Search task 3

- **Research and explain the difference between:**
  - **DROP CONSTRAINT**
  - **DISABLE CONSTRAINT**
- **Required:**
  - **Do both achieve the same goal?**
  - **What is the main difference between them?**
  - **When should we use DROP CONSTRAINT?**
  - **When should we use DISABLE CONSTRAINT?**
  - **Give practical database examples for each case.**

***1.1 DROP Constraint :-**

** Definition :-

- **is used to permanently (بشكل نهائي) remove a constraint from a table.**

After dropping:

- **The constraint no longer exists in the database.**
- **Oracle forgets the constraint definition completely.**

** Syntax / Example :-

- **here we create table student with two columns , and put CONSTRAINT to each column**

```oracle
CREATE TABLE students (
    student_id NUMBER CONSTRAINT students_id_pk PRIMARY KEY,
    email VARCHAR2(100) CONSTRAINT students_email_unique UNIQUE
);
```

- **Here we remove constraint (students_email_unique) from the table**

```oracle
ALTER TABLE students DROP CONSTRAINT students_email_unique;
```

** Result from this example :-

- **The constraint is deleted permanently (بشكل نهائي).**
- **Duplicate values become allowed.**
- **To use the constraint again, you must recreate it .**
  عشان نستخدم ال constraint ده تاني لازم نعمله re-create من جديد لأنه خلاص حذفته بشكل نهائي من table

***1.2 DISABLE Constraint :-**

** Definition :-

- **temporarily (تعطيل وليس حذف نهائي) turns off a constraint without deleting it.**
- **هنا كأني بعطل ال constraint ده بشكل مؤقت بس من دون ما أعمله delete بشكل نهائي من database**

After disabling:

- **The constraint still exists. (هو لسه موجود وفي أي وقت أقدر أفعله تاني)**
- **Oracle stops checking the rule temporarily.**

** Syntax / Example :-

- **here we create table student with two columns , and put CONSTRAINT to each column**

```
CREATE TABLE students (
    student_id NUMBER CONSTRAINT students_id_pk PRIMARY KEY,
    email VARCHAR2(100) CONSTRAINT students_email_unique UNIQUE
);
```

- **Here we disable constraint (students_email_unique) from the table So , Now duplicate values can be inserted temporarily.**
  - **بعد ال disable , فممكن بشكل مؤقت إني أعمل insert ل duplicate values في column**

```
ALTER TABLE students DISABLE CONSTRAINT students_email_unique;
```

- **To enable it again :**
  - **هنا فعلناه تاني علي table**

```
ALTER TABLE students ENABLE CONSTRAINT students_email_unique;
```

** Result from this example :-

- **The constraint is not deleted permanently (بشكل نهائي)**
- **To use the constraint again, we only enable it without recreate it again .**

---

*2- **Do Both Achieve the Same Goal? ?*

- **Partially Yes ✔️ , ❌ But They Are NOT the Same**

✔️ ***2.1 Partially Yes :-*

- **Both can stop constraint validation temporarily or permanently.**

For example:

- Both may allow duplicate values after execution.

❌ ***2.2 But They Are NOT the Same:-*

- **The main difference is:**

DROP CONSTRAINT	DISABLE CONSTRAINT
Removes the constraint permanently	**Temporarily turns off the constraint**
Constraint definition is deleted	**Constraint definition remains stored**
Cannot enable again directly	**Can enable again بسهولة**
Need to recreate constraint	**No need to recreate**

---

*3- **When should we use DROP CONSTRAINT ?*

Use it when:

- The constraint is no longer needed.
- Database design has changed.

***3.1 Practical Example :-**

Suppose / Scenario :

- **In the beginning, each customer must have a unique phone number.**
- **Later, business rules changed and multiple customers are allowed to use the same phone number.**
- **So we remove the UNIQUE constraint permanently using DROP CONSTRAINT.**

```oracle
/*
 * ============================================================
 * Full Example : DROP UNIQUE Constraint
 * ============================================================

/* ============================================================
   STEP 1 : Create customers table
   ============================================================ */

CREATE TABLE customers (
    customer_id    NUMBER CONSTRAINT customers_id_pk PRIMARY KEY,

    first_name     VARCHAR2(50),

    phone_number   VARCHAR2(20) CONSTRAINT customer_phone_uk UNIQUE
);

/* ============================================================
   STEP 2 : Insert valid rows
   ============================================================ */

INSERT INTO customers VALUES (1, 'Ahmed', '01012345678');

INSERT INTO customers VALUES (2, 'Ali', '01099999999');


/* ============================================================
   STEP 3 : Try inserting duplicate phone number
   ============================================================ */

-- This will give ERROR
-- because phone_number must be unique
INSERT INTO customers VALUES (3, 'Sara', '01012345678');


/* ============================================================
   STEP 4 : Business rules changed
   ============================================================ */

-- Now multiple customers can share the same phone number

ALTER TABLE customers DROP CONSTRAINT customer_phone_uk;

/* ============================================================
   STEP 5 : Insert duplicate phone number again
   ============================================================ */
```

```sql
-- Now Oracle allows duplicate phone numbers
-- because the UNIQUE constraint was removed

INSERT INTO customers VALUES (3, 'Sara', '01012345678');

/* ============================================================
   STEP 6 : Display all records
   ============================================================ */

SELECT * FROM customers;


/*
 * ============================================================
 * Important Note:
 * ============================================================
 *
 * DROP CONSTRAINT permanently removes the constraint.
 *
 * After dropping:
 * - Oracle no longer checks uniqueness
 * - Duplicate values become allowed
 * - The constraint cannot be enabled again directly
 * - To use it again, it must be recreated
 *
 * ============================================================
 */
```

4- **When should we use DISABLE CONSTRAINT ?

Use it when:

- **You want to stop validation temporarily..**
- **During bulk inserts or updates operations**
- **Improving insert performance temporarily**

**3.1 Practical Example :-

Suppose / Scenario:

- **We have two tables customers, orders.**
- **The orders table contains a FOREIGN KEY constraint connected to the customers table.**
- **During large data import operations, FOREIGN KEY validation may slow down performance**
- **So we temporarily DISABLE the constraint, import the data, then ENABLE it again.**

```sql
/*
 * ============================================================
 * Example: Using DISABLE CONSTRAINT During Data Import
 * ============================================================

/* ============================================================
   STEP 1 : Create customers table
   ============================================================ */
```

```oracle
CREATE TABLE customers (
    customer_id   NUMBER CONSTRAINT customers_id_pk PRIMARY KEY,
    customer_name VARCHAR2(100)
);


/* ==============================================================
   STEP 2 : Create orders table with FOREIGN KEY
   ============================================================== */

CREATE TABLE orders (
    order_id      NUMBER CONSTRAINT orders_id_pk PRIMARY KEY,
    order_date    DATE,
    customer_id   NUMBER,

    CONSTRAINT orders_customer_fk FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
);

/* ==============================================================
   STEP 3 : Insert data into customers table
   ============================================================== */

INSERT INTO customers VALUES (1, 'Ahmed');
INSERT INTO customers VALUES (2, 'Ali');

/* ==============================================================
   STEP 4 : Insert valid order records
   ============================================================== */

INSERT INTO orders VALUES (101, SYSDATE, 1);
INSERT INTO orders VALUES (102, SYSDATE, 2);


/* ==============================================================
   STEP 5 : Disable FOREIGN KEY constraint temporarily
   ============================================================== */

ALTER TABLE orders DISABLE CONSTRAINT orders_customer_fk;

/* ==============================================================
   STEP 6 : Insert data while constraint is disabled
   ============================================================== */

-- This customer_id = 999 does NOT exist in customers table
-- But Oracle allows it because the constraint is disabled

INSERT INTO orders VALUES (103, SYSDATE, 999);


/* ==============================================================
   STEP 7 : Enable constraint again
   ============================================================== */

ALTER TABLE orders ENABLE CONSTRAINT orders_customer_fk;
```

```
/*
 * ==============================================================
 * Important Note:
 * ==============================================================
 *
 * While the constraint is DISABLED:
 * - Oracle does not validate the FOREIGN KEY relationship.
 * - Invalid data can be inserted.
 *
 * After ENABLE:
 * - Oracle starts validating again.
 * - Future inserts must follow the FOREIGN KEY rule.
 *
 */
```